

SUNUM REHBERİ — Hastane Otomasyonu Dönem Projesi

Sunucu: (Adınız)

Danışman: Arş. Gör. Safa Enes TÜRKOĞLU · Oda 308

Süre: ~15–20 dakika



Sunu Akışı

1. Giriş ve Proje Tanıtımı	(2 dk)
2. Veritabanı Şeması Gösterimi	(3 dk)
3. Canlı Demo - Sekreter Paneli	(5 dk)
4. Canlı Demo - Doktor/Hasta	(3 dk)
5. İleri SQL Yapıları Açıklaması	(5 dk)
6. Sorular ve Kapanış	(2 dk)

Slayt 1 — Proje Tanıtımı

Söylenecekler:

"Bu proje, bir hastane otomasyon sisteminin veritabanı ve arayüz katmanını kapsıyor. SQL Server LocalDB üzerinde 15 ilişkili tablo, C# WinForms arayüzü ve ADO.NET kullandım. Hasta, doktor ve sekreter olmak üzere 3 kullanıcı rolü var."

Ekranda göster: Ana giriş ekranı (main_login_form)

Slayt 2 — Veritabanı Şeması

Söylenecekler:

"15 tablo ve 15 yabancı anahtar ilişkisi var. Temel zincir şu şekilde: Poliklinik → Branş → Oda → Doktor → Randevu → Reçete → İlaç. Ayrıca log, ödeme ve tetkik tabloları bu zincire bağlanıyor."

Ekranda göster: [veritabani_semasi.md](#) — ER Diyagramı

Vurgula: - Table_recete_ilac → M:N ara tablo örneği - Log tabloları trigger ile otomatik dolduruluyor - Table_odeme → her randevuya otomatik ödeme kaydı

Slayt 3 — Canlı Demo: Sekreter Paneli (CRUD)

Adım 1 — Giriş

- TC: 33333333333 · Şifre: 1234 ile sekreter olarak gir
- Dashboard'daki **Stat Kartlarını** göster (View'dan geliyor)

Adım 2 — READ (Listeleme)

- Sol panelde Branş listesi görünüyor
- Doktor listesi görünüyor
- **"Randevu Listesi"** butonuna tıkla → `vw_RandevuSunum` view'ı

Adım 3 — CREATE (Randevu Oluşturma)

- Ortadaki formu doldur: tarih, saat, branş, doktor seç
- **"Kaydet"** tıkla → `sp_RandevuOlustur` çalışır (Transaction!)
- Stat kartlarının güncellediğini göster

Adım 4 — DELETE (Randevu Silme)

- Randevu ID gir, **"Randevuyu Sil"** tıkla
- `sp_RandevuSil` çalışır

Adım 5 — Duyuru CREATE

- Duyuru metnini gir, **"Oluştur"** tıkla

Slayt 4 — Canlı Demo: Doktor Paneli (CRUD + Bilgi Güncelleme)

Adım 1 — Doktor Girişi

- TC: 11111111111 · Şifre: 1234
- Randevularını görüyor, şikayetleri okuyabiliyor

Adım 2 — Bilgi Güncelleme

- **"Bilgi Güncelle"** linkine tıkla
- Ad/Soyad/Şifre/Branş değiştir → **"Güncelle"** → `sp_DoktorGuncelle`
- **Trigger çalıştı!** → Log tablosuna kayıt gitti

Adım 3 — Sekreter'den Doktor CRUD

- Sekreter → "Doktor Listesi"
 - Yeni doktor ekle → `sp_DoktorKaydet`
 - Sil butonu → `sp_DoktorSil`
-

Slayt 5 — Canlı Demo: Hasta Paneli

Adım 1 — Hasta Girişi

- TC: 22222222222 · Şifre: 1234
- Özet kartlar: kendi randevu sayısı, boş kontenjan, duyuru

Adım 2 — Randevu Alma (Transaction Demo)

- Branş seç → Doktor seç → Şikayet yaz
- Boş slot'a çift tıkla → "Randevu Al"
- `sp_HastaRandevuAl` → Transaction içinde çalışır
- Dolu olan slota tıklamayı dene → `RAISERROR` ile hata döner, Rollback!

Adım 3 — Hasta Bilgi Güncelleme

- "Bilgi Güncelle" → `sp_HastaGuncelle`
-

Slayt 6 — İleri SQL Yapıları (Kod Gösterimi)

VIEW Örneği

```
-- vw_YonetimOzeti - Sekreter stat kartlarına veri sağlar
SELECT
    (SELECT COUNT(*) FROM Table_hasta)          AS toplam_hasta,
    (SELECT COUNT(*) FROM Table_doktor)         AS toplam_doktor,
    (SELECT COUNT(*) FROM Table_brans)          AS toplam_brans,
    ...
```

TRIGGER Örneği

```
-- trg_Doktor_Log - Doktor tablosu her değiştiğinde otomatik çalışır
CREATE TRIGGER trg_Doktor_Log ON Table_doktor
AFTER INSERT, UPDATE, DELETE AS
BEGIN
    INSERT INTO Table_kullanici_log (tablo_adi, islem_turu, ...)
    SELECT 'Table_doktor',
```

```
        CASE WHEN EXISTS(SELECT 1 FROM inserted) AND EXISTS(SELECT 1
FROM deleted)
            THEN 'UPDATE' ... END, ...
END
```

TRANSACTION Örneği

```
-- sp_HastaRandevuAl - Randevu dolu ise hata, değilse al
BEGIN TRANSACTION;
BEGIN TRY
    IF EXISTS (... randevu_durum = 1)
        RAISERROR('Secilen randevu dolu.', 16, 1);
    UPDATE Table_randevu SET hasta_tc = @hasta_tc, randevu_durum = 1 ...
    INSERT INTO Table_odeme ...
    COMMIT TRANSACTION;
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION; THROW;
END CATCH
```

FUNCTION Örneği

```
-- fn_HastaRandevuSayisi - View içinde hesaplama için kullanılır
CREATE FUNCTION fn_HastaRandevuSayisi(@hastaTc CHAR(11))
RETURNS INT AS BEGIN
    RETURN (SELECT COUNT(*) FROM Table_randevu WHERE hasta_tc = @hastaTc)
END
```

Slayt 7 — Yönetim Özeti (Log Gösterimi)

- Sekreter paneli → "Sunum Özeti" butonu
- vw_RandevuSunum view sonuçları tablo olarak
- Table_kullanici_log — tetikleyicilerin yazdığı kayıtlar
- "Veri bütünlüğü sağlanmış, hareketler izlenebilir" vurgusu



Değerlendirme Kriterleri Kontrol

Kriter	Nerede Gösterilir
CRUD — Create	Hasta kayıt, doktor ekle, randevu oluştur, duyuru ekle

Kriter	Nerede Gösterilir
CRUD — Read	Randevu listesi, doktor listesi, yönetim özeti
CRUD — Update	Hasta bilgi güncelle, doktor bilgi güncelle
CRUD — Delete	Doktor sil, randevu sil, branş sil
View	vw_YonetimOzeti stat kartları, vw_RandevuSunum
Trigger	Doktor güncelle → log otomatik yazıldı
Transaction	Dolu randevuya tıkla → rollback göster
Function	vw_HastaRandevuGecmisi'nde fn_HastaRandevuSayisi
Stored Procedure	Her CRUD butonu bir SP çağırıyor
Log Tabloları	Yönetim özeti ekranında görülebilir
10+ İlişkili Tablo	ER Diyagramı — 15 tablo



Sunuma Çıktı Alınacaklar

1. **Bu sunum rehberi** (kısaltılmış hali)
2. **Veritabanı şeması** ([veritabanı semasi.md](#)) — ER diyagramı ekran görüntüsü
3. **Proje raporu** ([proje raporu.md](#)) — baskıya hazır

[!TIP] Sunumda önce şemayı göster, sonra canlı demo yap. "Şimdi bunu kodda göstereyim" geçişini kullan.

[!IMPORTANT] Randevu alma Transaction demosu çok etkileyici — dolu slotu tıklayıp rollback'i canlı göster!